Neural Networks with Recurrent Generative Feedback

Yujia Huang
Caltech

James Gornet
Caltech

Sihui Dai
Caltech

Zhiding Yu
NVIDIA

Tan Nguyen
Rice University

Doris Y. Tsao
Caltech

Anima Anandkumar
Caltech / NVIDIA

Abstract

Neural networks are vulnerable to input perturbations such as additive noise and adversarial attacks. In contrast, human perception is much more robust to such perturbations. The Bayesian brain hypothesis states that human brains use an internal generative model to update the posterior beliefs of the sensory input. This mechanism can be interpreted as a form of self-consistency between the maximum a posteriori (MAP) estimation of the internal generative model and the external environmental. Inspired by this framework, we enforce consistency in neural networks by incorporating generative recurrent feedback. We implement this framework on convolutional neural networks (CNNs). The proposed framework, Convolutional Neural Networks with Feedback (CNN-F), introduces a generative feedback with latent variables into existing CNN architectures, making consistent predictions via alternating MAP inference under a Bayesian framework. CNN-F shows considerably better adversarial robustness over regular feedforward CNNs on standard benchmarks. In addition, With higher V4 and IT neural predictivity, CNN-F produces object representations closer to primate vision than conventional CNNs.

1 Introduction

Vulnerability in feedforward neural networks Conventional deep neural networks (DNNs) often contain many layers of feedforward connections. With the ever-growing network capacities and representation abilities, they have achieved great success. For example, recent convolutional neural networks (CNNs) have impressive accuracy on large scale image classification benchmarks [31]. However, current CNN models also have significant limitations. For instance, they can suffer significant performance drop from corruptions which barely influence human recognition [4]. Studies also show that CNNs can be misled by imperceptible noise known as adversarial attacks [30].

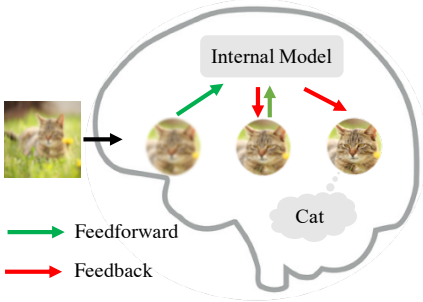


Figure 1: An intuitive illustration of recurrent generative feedback in human visual perception system.

Feedback in the human brain To address the weaknesses of CNNs, we can take inspiration from of how human visual recognition works, and incorporate certain mechanisms into the CNN design. While human visual cortex has hierarchical feedforward connections, backward connections from higher level to lower level cortical areas are something that current artificial networks are lacking [7]. Studies suggest these backward connections carry out top-down processing which improves the representation of sensory input [16]. In addition, evidence suggests recurrent feedback in the human visual cortex is crucial for robust object recognition. For

Preprint. Under review.

example, humans require recurrent feedback to recognize challenging images [12]. Obfuscated images can fool humans without recurrent feedback [6]. Figure 1 shows an intuitive example of recovering a sharpened cat from a blurry cat and achieving consistent predictions after several iterations.

Predictive coding Computational neuroscientists speculate that Bayesian inference models human perception [15]. Predictive coding, for example, is a specific formulation of hierarchical Bayesian inference that assumes Gaussian distributions on all variables [25]. Predictive coding uses recurrent, feedback pathways to perform Bayesian inference. According to predictive coding theory, feedback pathways encode predictions of lower level inputs. The residual errors are used recurrently to update the predictions.

Summary of results In this paper, we extend the principle of predictive coding to explicitly incorporate Bayesian inference in neural networks via generative feedback connections. We then investigate whether feedback promotes robust object recognition. Our contributions are as follows:

Generative feedback Inspired by the Bayesian brain hypothesis and predictive coding theory, we hypothesize that the feedback process reconstructs the stimulus using an internal generative model of the world. To generate images from a low dimensional label, we also add appropriate auxiliaries that capture variation in the images. Specifically, we adopt a recently proposed model, named the Deconvolutional Generative Model (DGM) [24], as the generative feedback (Figure 2). The DGM uses hierarchical latent variables to generate images. We show that Bayesian inference in the DGM is achieved by CNN with adaptive nonlinear operators.

Self-consistency We introduce generative feedback to NN and impose self-consistency to enable robustness. Intuitively, self-consistency means that the label, auxillary information and images should be consistent with each other. Furthermore, our internal model of the world should be consistent with the external stimuli. Mathematically, we use a generative model to describe the joint distribution of labels, latent variables and features of input images. The predicted label, latent variables and features are maximum a posteriori (MAP) estimates conditioned on the other two elements. They are self-consistent if the MAP estimates are consistent with each other (Figure 4).

CNN with Feedback (CNN-F) We incorporate generative recurrent feedback into CNN and term this model as CNN-F. We impose self-consistency in CNN-F by performing alternating MAP inference for the label, latent variables and image features. MAP inference for the label and image features given latent variables is straightforward from classification and generation networks. MAP inference for the hierarchical latent variables is approximated by iterated conditional modes (ICM) [3], where we iteratively compute MAP estimates for latent variables at one layer conditioning on all other layers. This leads to recurrent generative feedback to feedforward layers in CNN-F (Figure 2).

Adversarial robustness We evaluate the adversarial robustness of CNN-F on MNIST and Fashion-MNIST datasets. CNN-F achieves significantly better adversarial accuracy than CNN under both standard training and adversarial training. Furthermore, training and evaluating CNN-F with more

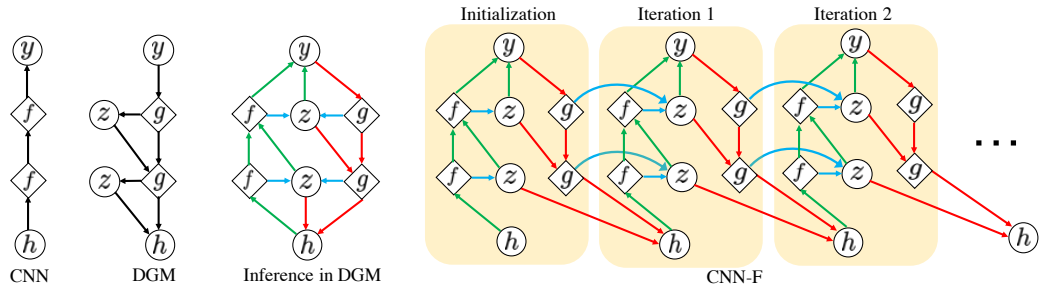


Figure 2: **Left: CNN, Graphical model for the DGM and the inference network for the DGM.** We use the DGM to as the generative model for the joint distribution of image features h, labels y and latent variables z. MAP inference for h, y and z is denoted in red, green and blue respectively. f and g denotes feedforward features and feedback features respectively. **Right: CNN with feedback (CNN-F)**. CNN-F performs alternating MAP inference via recurrent feedforward and feedback pathways to enforce self-consistency.

iterations both help improve robustness, indicating that recurrent feedback is crucial for recognizing challenging images.

Biological plausibility Given the lateral and backward connections in the primate brain, we investigate whether the CNN-F's generative feedback produces more biologically similar neural networks. Trained on ImageNet-12, we show that the CNN-F has higher V4 and IT neural predictivity compared to its corresponding CNN. This demonstrates that the CNN-F models human vision significantly closer compared to CNNs.

2 Approach

In this section, we first formally define self-consistency. Then we give a specific form of generative feedback in CNN and impose self-consistency on it. We term this model as CNN-F. Finally we show the training and testing procedure in CNN-F. Throughout, we use the following notations:

Let $x \in \mathbb{R}^n$ be the input of a neural network and $y \in \mathbb{R}^K$ be the output. In image classification problems, x is image and $y = (y^{(1)}, \ldots, y^{(K)})$ is one-hot encoded label. K is the total number of classes. K is usually much less than n. We use L to denote the total number of layers of the network, and index the input layer to the feedforward network as layer 0. Let $h \in \mathbb{R}^m$ be encoded feature of x at layer k of the feedforward pathway. Feedforward pathway computes feature map $f(\ell)$ from layer 0 to layer L, and feedback pathway generates $g(\ell)$ from layer L to k. $g(\ell)$ and $f(\ell)$ have the same dimensions. To generate h from y, we introduce latent variables for each layer of CNN. Let $z(\ell) \in \mathbb{R}^{C \times H \times W}$ be latent variables at layer ℓ, where C, H, W are the number of channels, height and width for the corresponding feature map. Finally, we use $p(h, y, z; \theta)$ to denote the joint distribution parameterized by θ. θ includes the weight W of convolutional and fully connected layers and the bias term b. We use $\hat{h}$, $\hat{y}$ and $\hat{z}$ to denote the MAP estimates of h, y, z conditioning on the other two variables.

2.1 Generative feedback and Self-consistency

Human brain and NN are similar in having a hierarchical structure. In human visual perception, external stimuli are first preprocessed by lateral geniculate nucleus (LGN) and then sent to be processed by V1, V2, V4 and Inferior Temporal (IT) cortex in the ventral cortical visual system. Conventional NN use feedforward layers to model this process and learn a one-direction mapping from input to output. However, numerous studies suggest that in addition to the feedforward connections from V1 to IT, there are feedback connections among these cortical areas [7].

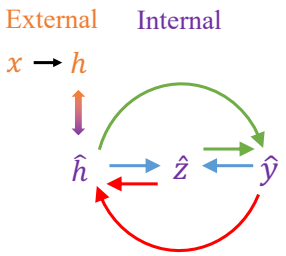


Figure 4: Self-consistency among $\hat{h}, \hat{z}, \hat{y}$ and consistency between $\hat{h}$ and h.

Inspired by the Bayesian brain hypothesis and the predictive coding theory, we propose to add generative feedback connections to NN. Since h is usually of much higher dimension than y, we introduce latent variables z to account for the information loss in the feedforward process. We then propose to model the feedback connections as MAP estimation from an internal generative model that describes the joint distribution of h, z and y. Furthermore, we realize recurrent feedback by imposing self-consistency (Definition 2.1).

Definition 2.1. (Self-consistency) Given a joint distribution $p(h, y, z; \theta)$ parameterized by θ, $(\hat{h}, \hat{y}, \hat{z})$ are self-consistent if they satisfy the following constraints:

$$\hat{y} = \arg\max_y p(y|\hat{h}, \hat{z}), \qquad \hat{h} = \arg\max_h p(h|\hat{y}, \hat{z}), \qquad \hat{z} = \arg\max_z p(z|\hat{h}, \hat{y}) \tag{1}$$

In words, self-consistency means that MAP estimates from an internal generative model are consistent with each other. In addition to self-consistency, we also impose the consistency constraint between $\hat{h}$ and the external input features (Figure 4). We hypothesize that for *easy* images (familiar images to human, clean images in the training dataset for NN), the $\hat{y}$ from the first feedforward pass should automatically satisfy the self-consistent constraints. Therefore, feedback need not be triggered. For *challenging* images (unfamiliar images to human, unseen perturbed images for NN), recurrent feedback is needed to obtain self-consistent $(\hat{h}, \hat{y}, \hat{z})$ and to match $\hat{h}$ with h. This recurrence accounts for the dynamics in neural circuits [13] and the longer time that people need to process challenging images [12].

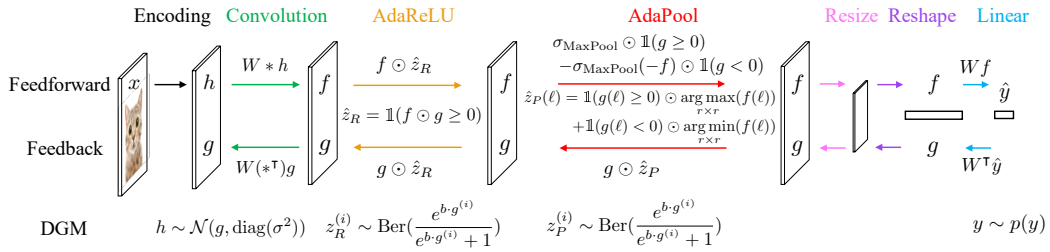


Figure 3: **Feedforward and feedback pathway in CNN-F.** $\hat{y}$ and $\hat{z}$ are computed by the feedforward pathway and $\hat{h}$ is computed from the feedback pathway.

2.2 Generative Feedback in CNN-F

CNN have been used to model the hierarchical structure of human retinatopic fields [5, 11], and have achieved state-of-the-art performance in image classification. Therefore, we introduce generative feedback to CNN and impose self-consistency on it. We term the resulting model as CNN-F.

We choose to use the DGM [24] as generative feedback in the CNN-F. The DGM introduces hierarchical binary latent variables and generates images from coarse to fine details. The generation process in the DGM is shown in Figure 3. First, y is sampled from the label distribution. Then each entry of $z(\ell)$ is sampled from a Bernoulli distribution parameterized by $g(\ell)$ and a bias term $b(\ell)$. $g(\ell)$ and $z(\ell)$ are then used to generate the layer below:

$$g(\ell - 1) = W(*^\intercal)(\ell)(z(\ell) \odot g(\ell)) \tag{2}$$

In this paper, we assume $p(y)$ to be uniform, which is realistic under the balanced label scenario. We assume that x follows Gaussian distribution centered at $g(0)$ with standard deviation σ.

2.3 Recurrence in CNN-F

In this section, we show that self-consistent $(\hat{h}, \hat{y}, \hat{z})$ in the DGM can be obtained via alternately propagating along feedforward and feedback pathway in CNN-F.

Feedforward and feedback pathway in CNN-F The feedback pathway in CNN-F takes the same form as the generation process in the DGM (Equation 2). The feedforward pathway in CNN-F takes the same form as CNN except for the nonlinear operators. In conventional CNN, nonlinear operators are $\sigma_{\text{ReLU}}(f) = \max(f, 0)$ and $\sigma_{\text{MaxPool}}(f) = \max_{r \times r} f$, where r is the dimension of the pooling region in the feature map (typically equals to 2 or 3). In contrast, we use σ_{AdaReLU} and σ_{AdaPool} given in Equation 14 in the feedforward pathway of CNN-F. These operators adaptively choose how to activate the feedforward feature map based on the sign of the feedback feature map. The feedforward pathway computes $f(\ell)$ using the recursion $f(\ell) = W(\ell) * \sigma(f(\ell - 1))\} + b(\ell)$ [1].

$$\sigma_{\text{AdaReLU}}(f) = \begin{cases} \sigma_{\text{ReLU}}(f), & \text{if } g \geq 0 \\ \sigma_{\text{ReLU}}(-f), & \text{if } g < 0 \end{cases} \qquad \sigma_{\text{AdaPool}}(f) = \begin{cases} \sigma_{\text{MaxPool}}(f), & \text{if } g \geq 0 \\ -\sigma_{\text{MaxPool}}(-f), & \text{if } g < 0 \end{cases} \tag{3}$$

MAP inference in the DGM We present MAP inference for h, y, z in the DGM in Theorem 2.1. First, we define generative classifier as a neural network that outputs the MAP distribution of y in a generative model. A well known example is that logistic regression is the generative classifier derived from Gaussian Naive Bayes model, where y is Boolean variable modeled by a Bernoulli distribution and $p(x|y)$ is assumed to follow Gaussian distribution. We use z_R and z_P to denote latent variables that are at a layer followed by AdaReLU and AdaPool respectively. $\mathbb{1}(\cdot)$ denotes indicator function. To ease MAP inference in the DGM, we have the following assumptions:

Assumption 2.1. **A.** The generated image at layer k from the DGM $g(k)$ has a constant ℓ_2 norm: $||g(k)||_2^2 = \text{const}$.

B. Prior distribution on the label is a uniform distribution: $p(y) = \text{const}$.

C. Normalization factor in $p(z|y)$ for each category is constant: $\sum_z e^{\eta(y,z)} = \text{const}$.

Theorem 2.1 (MAP inference in the DGM)**.** Under Assumption 2.1, the followings hold:

A. Let h be the feature at layer k, then $\hat{h} = g(k)$.

B. CNN with σ_{AdaReLU} and σ_{AdaPool} is the generative classifier derived from the DGM.

[1] σ takes the form of σ_{AdaPool} or σ_{AdaReLU}.

C. MAP estimate of $z(\ell)$ conditioned on h, y and $\{z(j)\}_{j\neq\ell}$ in the DGM is:

$$\hat{z}_R(\ell) = \mathbb{1}(\sigma_{\text{AdaReLU}}(f(\ell)) \geq 0) \tag{4}$$

$$\hat{z}_P(\ell) = \mathbb{1}(g(\ell) \geq 0) \odot \arg\max_{r\times r}(f(\ell)) + \mathbb{1}(g(\ell) < 0) \odot \arg\min_{r\times r}(f(\ell)) \tag{5}$$

Proof. For part A, we have $\hat{h} = \arg\max_h p(h|\hat{y}, \hat{z}) = \arg\max_h p(h|g(k)) = g(k)$. The second equality is obtained because $g(k)$ is a deterministic function of $\hat{y}$ and $\hat{z}$. The third equality is obtained because $h \sim \mathcal{N}(g(k), \text{diag}(\sigma^2))$.
For part B and C, please refer to Appendix A. □

Remark. Theorem 2.1.A and B show that $\hat{h}$ is the generated feature map at bottom level in CNN-F and $\hat{y}$ is the output from the feedfoward pathway.
Theorem 2.1.C states that $\hat{z}_R = 1$ if the sign of the feedforward feature map matches with that of the feedback feature map. $\hat{z}_P = 1$ at locations that satisfy one of these two requirements: 1) the value in the feedback feature map is non-negative and it is the maximum value within the local pooling region or 2) the value in the feedback feature map is negative and it is the minimum value within the local pooling region. Using Theorem 2.1.C, we approximate $\{\hat{z}(\ell)\}_{\ell=1:L}$ by greedily finding the MAP estimate of $\hat{z}(\ell)$ conditioning on all other layers.

Iterative inference in CNN-F To impose self-consistency on $(\hat{h}, \hat{y}, \hat{z})$, we solve the optimization problem specified in 1 via alternating optimization. The resulting iterative inference procedure in CNN-F is as follows (Figure 2, right). In the initialization step, image x is passed through a standard CNN, and latent variables are initialized using Equation 18 and 19 with conventional σ_{ReLU} and σ_{MaxPool}. Then, the feedback generative network uses $\hat{y}_0$ and $\{\hat{z}_0(\ell)\}_{\ell=1:L}$ to generate intermediate features $\{g_0(\ell)\}_{\ell=1:L}$, where the subscript denotes the number of iterations. In practice, we use logits instead of one-hot encoded label in the generative feedback to maintain uncertainty in each category. We assign $g(k)$ to the input features for the next iteration $\hat{h}_1$, where k is the layer that we reconstruct to in the feedback pathway. $\hat{h}_1$ is then fed back to the feedforward pathway for the next iteration. Starting from the first iteration, we use σ_{AdaReLU} and σ_{AdaPool} instead of conventional σ_{ReLU} and and σ_{MaxPool} in the feedforward pathway to infer $\hat{z}$ (Theroem 2.1). This iterative inference procedure is described in Algorithm 1.

Algorithm 1: Iterative inference in CNN-F

Input : Input image x, index k of the target layer to reconstruct, maximum number of iterations N.
Initialize $\{\hat{z}(\ell)\}_{\ell=1:L}$ by standard the CNN;
while $t < N$ **do**
  Feedback pathway: generate $g_t(k)$ from $\hat{y}_t$ and $\hat{z}_t(\ell)$, $\ell = k, \ldots, L$.;
  Feedforward pathway: predict $\hat{y}_{t+1}$ from $g_t(k)$ and compute $\hat{z}_{t+1}(\ell)$, $\ell = k, \ldots, L$.;
end
return $\hat{h}_N, \hat{y}_N, \hat{z}_N$

2.4 Training and testing in CNN-F

During training, we have three goals: 1) train a generative model to model the data distribution, 2) train a generative classifier and 3) enforce self-consistency in the model. We first approximate self-consistent $(\hat{h}, \hat{y}, \hat{z})$ and then update model parameters based on the losses listed in Table 1. Each loss term is computed for every iteration. Minimizing the reconstruction loss and conditional latent likelihood loss is equivalent to maximizing the log likelihood of $(h, \hat{y}_t, \hat{z}_t)$. Minimizing the reconstruction loss also improves the consistency between $\hat{h}_t$ and h. Minimizing the Cross entropy loss helps training the generative classifier.

During testing time, CNN-F finds self-consistent $(\hat{h}, \hat{y}, \hat{z})$ given the input image using iterative inference described in Algorithm 1.

Table 1: Training losses used to train CNN-F.

	Form	Purpose
Cross entropy loss	$\log p(y \mid \hat{h}_t, \hat{z}_t; \theta)$	classification
Reconstruction loss	$\log p(h \mid \hat{y}_t, \hat{z}_t; \theta) = \|\|h - \hat{h}\|\|_2^2.$	generation, self-consistency
Conditional latent likelihood loss	$\log p(\hat{z}_t \mid \hat{y}_t; \theta) = \log(\text{Softmax}(\eta(\hat{y}_t, \hat{z}_t)))$ [2]	generation

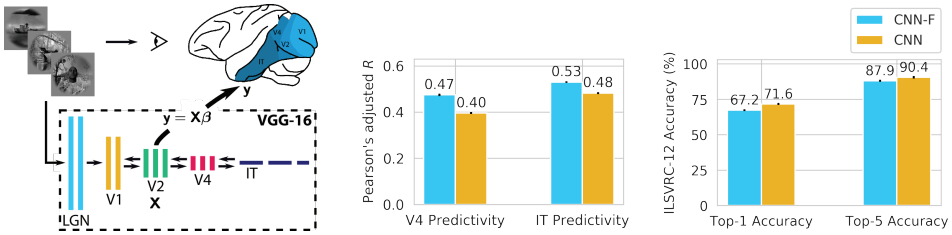


Figure 5: **The generative feedback in CNN-F models predict a significant portion of primate vision neural responses. Left** The Brain-Score's experimental paradigm presents visual image stimuli both to primates and a neural network. The neural similarity is measured by the correlation between the neural network's activations and the primate's neural responses. **Middle** With the VGG-16 architecture, the CNN-F's generative feedback increases the correspondence with the primate's V4 and IT neural responses. **Right** The CNN-F's generative feedback produces a drop in accuracy on ILSVRC-12 compared to the CNN.

3 CNN-F models correspond more to primate vision compared to a CNN

A CNN has a substantial correspondence with the primate visual cortex [27]. To measure this correspondence, neural predictivity is a benchmark that quantifies the similarity between an artificial response—such as a CNN's hidden layer activations—and a biological response—such as a neuronal activations. To measure similarity, neural predictivity linearly maps an CNN's hidden layer activations to the primate visual cortex's neural activations using a PLS regression model with 25 components [35]. The Pearson's r from this regression quantifies the neural predictivity.

Given the prevalence of backwards connections in the visual cortex, we investigated whether generative feedback increases the neural predictivity of a CNN with respect to the primate visual system. To investigate whether generative feedback produces object representations closer to primate vision, we trained a CNN-F and a CNN with the VGG-16 architecture on the ILSVRC-12 (ImageNet-2012) dataset [29]. We compared the biological correspondence of the CNN-F and CNN through the Brain-Score, which contains neural similarity benchmarks on V4 and IT (Figure 5).

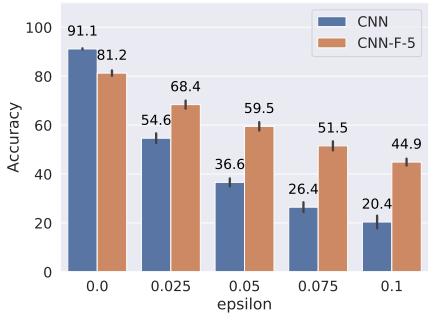
Figure 5 shows how the models perform on the Brain-Score and ImageNet. In both V4 and IT neural predictivity, the CNN-F has a greater correspondence with the primate brain compared to the CNN (V4: $t = 10.62$, $p = 7.11 \times 10^{-9}$; IT: $t = 7.89$, $p = 4.28 \times 10^{-7}$) with a 4.43% decrease in Top-1 ILSVRC-12 classification accuracy. This demonstrates that the CNN-F's generative feedback—with a CNN's convolutional layers—corresponds to a significant portion of primate vision neural responses.

4 CNN-F produces more robust object recognition

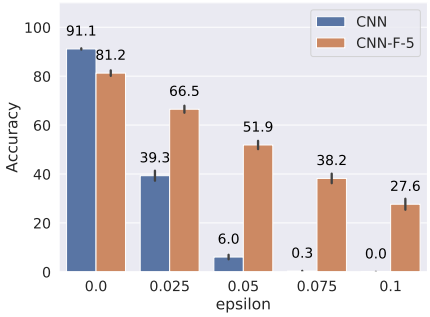
4.1 CNN-F is robust against adversarial attacks

Attack methods We explore various ways to attack the CNN-F. First, we attempt to perform an end-to-end BPDA [1] attack on CNN-F. Due to the approximation of non-differentiable activation operators and the depth of the unrolled CNN-F, the effectiveness of this attack degrades. Second, we attack the first feedforward pass of CNN-F. This is equivalent to attacking a CNN with the same parameters and attempting to transfer the attack from the CNN over to the CNN-F. We call this method transfer attack for short. Transfer attack overcomes the obfuscated gradient issue, and is more effective than end-to-end attack. Therefore, the adversarial accuracy we present here is against transfer attack on the cross entropy loss. We use the Fast Gradient Sign Attack Method (FGSM) [9] Projected Gradient Descent (PGD) method to attack. For PGD attack, we generate adversarial

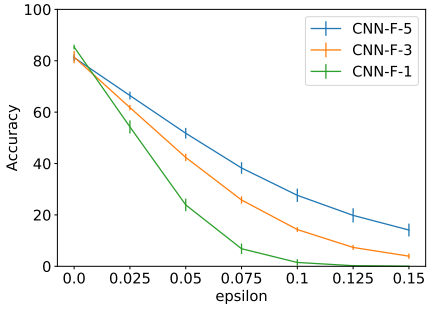
[2] According to the distribution specified by the DGM, $\eta(y, z) = \sum_{\ell=1}^{L} \frac{1}{\sigma^2} \langle b(\ell), z(\ell) \odot g(\ell) \rangle$, see Appendix A for details.



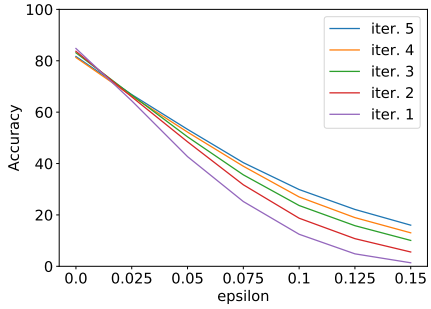
(a) Standard training. Testing w/ FGSM.



(b) Standard training. Testing w/ PGD-40.

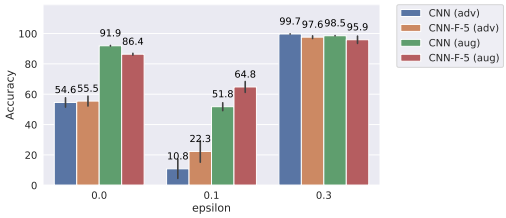


(c) Train CNN-F with different number of iterations. Testing w/ PGD-40.

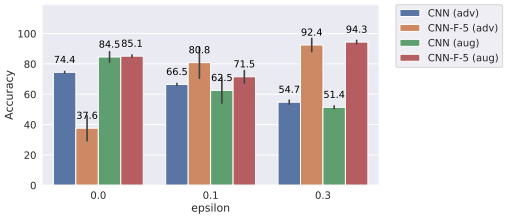


(d) Evaluate a CNN-F-5 model with various number of iterations against PGD-40 attack.

Figure 6: **Adversarial robustness of CNN-F with standard training.** CNN-F achieves higher accuracy on Fashion-MNIST than CNN under standard training on Fashion-MNIST. More iterations are needed for larger adversarial perturbation magnitude. CNN-F-k stands for CNN-F trained with k iterations; PGD-40 stands for a PGD attack with 40 steps.



(a) Adv. training w/ FGSM ($\epsilon = 0.3$). Testing w/ FGSM.

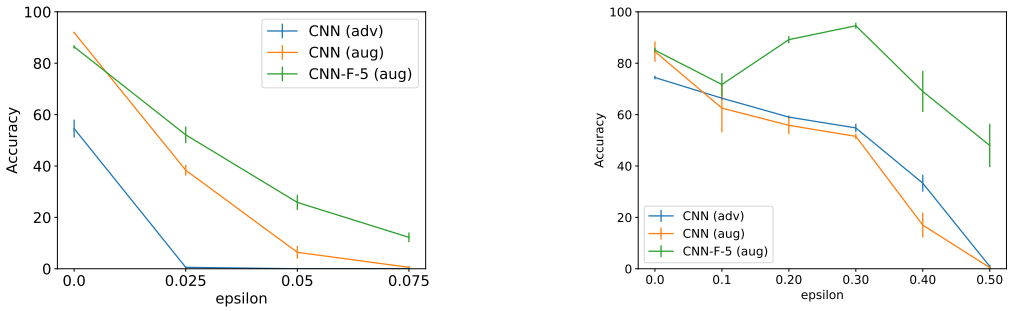


(b) Adv. training w/ PGD-40 ($\epsilon = 0.3$). Testing w/ PGD-40.

Figure 7: **Adversarial robustness of CNN-F with adversarial training.** CNN-F achieves higher adversarial accuracy and natural accuracy than CNN when trained with both clean and adversarial images on Fashion-MNIST.

samples within L_∞-norm constraint, and denote the maximum L_∞-norm between adversarial images and clean images as ϵ.

Standard Training We train a CNN-F model with two convolutional layers followed by two fully-connected layers. For training details and results on MNIST dataset, please refer to Appendix B. For all the figures in the paper, each accuracy is averaged over 5 runs and the error bar indicates standard deviation. We test the robustness of CNN-F under two settings: standard training on clean images and adversarial training [20], and show improvement of CNN-F compared to CNN in both settings. The results for standard training are shown in Figure 6. We see that CNN-F has considerably better robustness than CNN. Furthermore, training and evaluating CNN-F with more iterations both



(a) Adv. training w/ FGSM ($\epsilon = 0.3$). Testing w/ PGD-40.

(b) Adv. training w/ PGD-40 ($\epsilon = 0.3$). Testing w/ PGD-40.

Figure 8: **CNN-F generalize better to unseen attacks than CNN.** (a) Performance of CNN-F trained with FGSM adversarial samples against PGD-40 adversaries. (b) Performance of CNN-F trained with PGD-40 adversarial samples with $\epsilon = 0.3$ against adversaries with different strength.

help improve robustness, and we see larger improvements for higher ϵ (Figure 6c, 6d). This indicates that recurrent feedback is crucial for recognizing challenging images.

Adversarial Training Instead of train CNN-F solely on adversarial images as conventional adversarial training ([20]), we train CNN-F with both clean images and adversarial images in a data augmentation manner. We use cross entropy loss on both clean images and adversarial images, and we let the CNN-F to reconstruct adversarial samples to the corresponding clean images. We find that training CNN-F in this way mitigate the overfitting problem in conventional adversarial training. As shown in Figure 7b, CNN-F-5 trained with both clean images and adversarial images achieves high accuracy on both clean images and adversarial images. Empirically, we find that the optimal number of iterations depends on the *hardness* of images (See Appendix B.4). Therefore, we believe that a more fundamental solution to this overfitting problem is to adaptively choose number of iterations based on the *hardness* of input images. We save this as our future work.

Figure 7 shows that CNN-F further improves the robustness of CNN when trained with adversarial images. Furthermore, CNN-F generalizes better to unseen attacks compared to CNN. Figure 8a shows that CNN-F trained with FGSM adversarial images generalizes better to PGD-40 attacks compared to CNN. Figure 8b shows that CNN-F trained with PGD-40 attack suffers less from accuracy drop against adversaries with larger strength.

4.2 The generative feedback in CNN-F models restores perturbed images

Given that CNN-F models are robust to adversarial attacks, we examine the models' mechanism for robustness. Studies suggest that feedback in the visual cortex is crucial to robust object recognition [12, 6]. We investigate this principle with generative feedback in CNN-F models. We train a CNN-F model Fashion-MNIST. A validation image is then selected from Fashion-MNIST. Using the image's two largest principal components, a two-dimensional hyperplane $\subset \mathbb{R}^{28\times 28}$ intersects the image with the image at the center. Vector arrows visualize the generative feedback's perturbation on the hyperplane's position. In Figure 9(a), we find that generative feedback perturbs samples across decision boundaries toward the validation image. This demonstrates that the CNN-F's generative feedback can restore perturbed, distorted images to their uncorrupted objects.

We further explore this principle with regard to adversarial examples. The CNN-F model can correct initially wrong predictions. Figure 9(b) Grad-CAM activations visualize the network's attention from an incorrect prediction to a correct prediction on PGD-40 adversarial samples [28]. To correct predictions, the CNN-F model does not initially focus on specific features. Rather, it either identifies the entire object or the entire image. With generative feedback, the CNN-F begins to focus on specific features. This is reproduced in clean images as well as images corrupted by blurring and additive noise 9(c). Furthermore, with these perceptible corruptions, the CNN-F model can reconstruct the clean image with generative feedback 9(d). This demonstrates that the generative feedback is one mechanism that restores perturbed images.

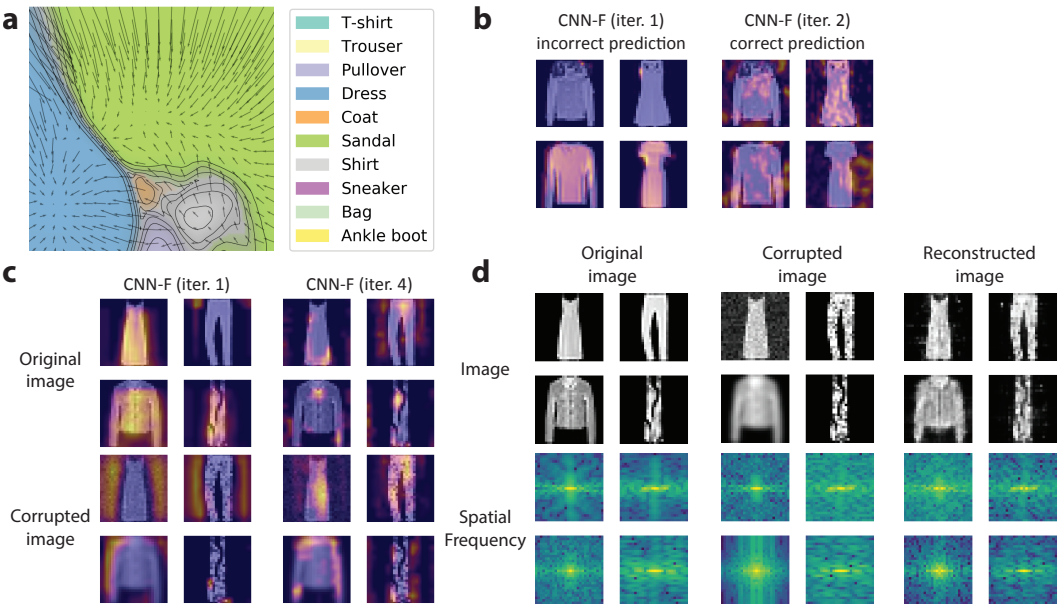


Figure 9: **The generative feedback in CNN-F models restores perturbed images. a**, The decision cell cross-sections for a CNN-F trained on Fashion-MNIST. Arrows visualize the feedback direction on the cross-section. **b**, Fashion-MNIST classification accuracy on PGD adversarial examples; Grad-CAM activations visualize the CNN-F model's attention from incorrect (iter. 1) to correct predictions (iter. 2). **c**, Grad-CAM activations across different feedback iterations in the CNN-F. **d**, From left to right: clean images, corrupted images, and images restored by the CNN-F's feedback.

5 Related work

Robust neural networks with latent variables Latent variable models are a unifying theme in robust neural networks. The consciousness prior [2] postulates that natural representations—such as language—operate in a low-dimensional space, which may restrict expressivity but also may facilitate rapid learning. If adversarial attack introduce examples outside this low-dimensional manifold, latent variable models can map these samples back to the manifold. A related mechanism for robustness is state reification [18]. Similar to self-consistency, state reification models the distribution of hidden states over the training data. It then maps less likely states to more likely states. MagNet and Denoising Feature Matching introduce similar mechanisms: using autoencoders on the input space to detect adversarial examples and restore them in the input space [21, 33]. Lastly, Defense-GAN proposes a generative adversarial network to approximate the data manifold [26]. CNN-F generalizes these themes into a Bayesian framework. Intuitively, CNN-F can be viewed as an autoencoder. In contrast to standard autoencoders, CNN-F requires stronger constraints through Bayes rule. CNN-F—through self-consistency—constrains the generated image to satisfy the *maximum a posteriori* on the predicted output.

Computational models of human vision Recurrent models and Bayesian inference have been two prevalent concepts in computational visual neuroscience. Recently, Kubilius et al. [17] proposed CORnet as a more accurate model of human vision by modeling recurrent cortical pathways. Like CNN-F, they show CORnet has a larger V4 and IT neural similarity compared to a CNN with similar weights. Linsley et al. [19] suggests hGRU as another recurrent model of vision. Distinct from other models, hGRU models lateral pathways in the visual cortex to global contextual information. While Bayesian inference is a candidate for visual perception, a Bayesian framework is absent in these models. The recursive cortical network (RCN) proposes a hierarchal conditional random field as a model for visual perception [8]. In contrast to neural networks, RCN uses belief propagation for both training and inference. With the representational ability of neural networks, we propose CNN-F to satisfy Bayesian inference with recurrent circuits in neural networks.

Feedback networks Feedback Network [36] uses convLSTM as building blocks and adds skip connections between different time steps. This architecture enables early prediction and enforces hierarchical structure in the label space. [23] ses architecture search to design local recurrent cells and

long range feedback to boost classification accuracy. [34] designs a bi-directional recurrent neural net, by recursively performing bottom up and top down computations, the model achieves more accurate and definitive image classification. Despite the promising progress on using feedback to improve classification accuracy on clean images, none of the previous works aim to improve classification robustness to the best of our knowledge.

Combining top-down and bottom-up signals in RNNs Mittal et al. [22] proposes combine attention and modularity mechanisms to route bottom-up (feedforward) and top-down (feedback) signals. They extend the Recurrent Independent Mechanisms (RIMs) [10] framework to a bidirectional structure such that each layer of the hierarchy can send information in both bottom-up direction and top-down direction. Our approach uses approximate Bayesian inference to provide top-down communication, which is more consistent with the Bayesian brain framework and predictive coding.

6 Conclusion

Inspired by the feedback connections in the brain, we propose to introduce recurrent generative feedback to neural networks. We instantiate the framework on CNN and term the model as CNN-F. We then demonstrate that the proposed feedback mechanism significantly improves the adversarial robustness in CNN. We visualized the dynamical behavior of CNN-F and shows its capability of restoring corrupted images. Furthermore, we find that the generative feedback of CNN-F predicts a significant portion of primate vision neural responses.

Acknowledgements

We thank Francisco Luongo and Haotao Wang for useful discussions. Y. Huang is supported by DARPA LwLL grants. J. Gornet is supported by supported by the NIH Predoctoral Training in Quantitative Neuroscience 1T32NS105595-01A1. A. Anandkumar is supported in part by Bren endowed chair, DARPA LwLL grants, Tianqiao and Chrissy Chen Institute for Neuroscience, Microsoft, Google, and Adobe faculty fellowships. D. T. Tsao is supported by Howard Hughes Medical Institute and Tianqiao and Chrissy Chen Institute for Neuroscience.

References

[1] A. Athalye, N. Carlini, and D. Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *ICML*, 2018.

[2] Y. Bengio. The consciousness prior. *arXiv:1709.08568*, 2019.

[3] J. Besag. On the statistical analysis of dirty pictures. *J. R. Statist. Soc. B*, 1986.

[4] S. Dodge and L. Karam. A study and comparison of human and deep learning recognition performance under visual distortions. In *ICCCN*, 2017.

[5] M. Eickenberg, A. Gramfort, G. Varoquaux, and B. Thirion. Seeing it all: Convolutional network layers map the function of the human visual system. *NeuroImage*, 2017.

[6] G. Elsayed, S. Shankar, B. Cheung, N. Papernot, A. Kurakin, I. Goodfellow, and J. Sohl-Dickstein. Adversarial examples that fool both computer vision and time-limited humans. In *NeurIPS*, 2018.

[7] D. J. Felleman and D. C. Van Essen. Distributed hierarchical processing in the primate cerebral cortex. *Cerebral Cortex*, 1991.

[8] D. George, W. Lehrach, K. Kansky, M. Lázaro-Gredilla, C. Laan, B. Marthi, X. Lou, Z. Meng, Y. Liu, H. Wang, A. Lavin, and D. S. Phoenix. A generative vision model that trains with high data efficiency and breaks text-based CAPTCHAs. *Science*, 2017.

[9] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples. In *ICLR*, 2015.

[10] A. Goyal, A. Lamb, J. Hoffmann, S. Sodhani, S. Levine, Y. Bengio, and B. Schölkopf. Recurrent independent mechanisms. *arXiv:1909.10893*, 2019.

[11] T. Horikawa and Y. Kamitani. Hierarchical neural representation of dreamed objects revealed by brain decoding with deep neural network features. *Front Comput Neurosci*, 2017.

[12] K. Kar, J. Kubilius, K. Schmidt, E. B. Issa, and J. J. DiCarlo. Evidence that recurrent circuits are critical to the ventral stream's execution of core object recognition behavior. *Nature neuroscience*, 2019.

[13] T. C. Kietzmann, C. J. Spoerer, L. K. Sörensen, R. M. Cichy, O. Hauk, and N. Kriegeskorte. Recurrence is required to capture the representational dynamics of the human visual system. *PNAS*, 2019.

[14] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.

[15] D. C. Knill and W. Richards. *Perception as Bayesian inference*. Cambridge University Press, 1996.

[16] P. Kok, J. F. Jehee, and F. P. De Lange. Less is more: expectation sharpens representations in the primary visual cortex. *Neuron*, 2012.

[17] J. Kubilius, M. Schrimpf, A. Nayebi, D. Bear, D. L. Yamins, and J. J. DiCarlo. Cornet: modeling the neural mechanisms of core object recognition. *bioRxiv preprint*, 2018.

[18] A. Lamb, J. Binas, A. Goyal, S. Subramanian, I. Mitliagkas, D. Kazakov, Y. Bengio, and M. C. Mozer. State-reification networks: Improving generalization by modeling the distribution of hidden representations. In *ICML*, 2019.

[19] D. Linsley, J. Kim, V. Veerabadran, C. Windolf, and T. Serre. Learning long-range spatial dependencies with horizontal gated recurrent units. In *NeurIPS*, 2018.

[20] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv:1706.06083*, 2017.

[21] D. Meng and H. Chen. Magnet: a two-pronged defense against adversarial examples. In *CCS*, 2017.

[22] S. Mittal, A. Lamb, A. Goyal, V. Voleti, M. Shanahan, G. Lajoie, M. Mozer, and Y. Bengio. Learning to combine top-down and bottom-up signals in recurrent neural networks with attention over modules. In *ICML*, 2020.

[23] A. Nayebi, D. Bear, J. Kubilius, K. Kar, S. Ganguli, D. Sussillo, J. J. DiCarlo, and D. L. Yamins. Task-driven convolutional recurrent models of the visual system. In *NeurIPS*, 2018.

[24] T. Nguyen, N. Ho, A. Patel, A. Anandkumar, M. I. Jordan, and R. G. Baraniuk. A bayesian perspective of convolutional neural networks through a deconvolutional generative model. *arXiv:1811.02657*.

[25] R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 1999.

[26] P. Samangouei, M. Kabkab, and R. Chellappa. Defense-gan: Protecting classifiers against adversarial attacks using generative models. In *ICLR*, 2018.

[27] M. Schrimpf, J. Kubilius, H. Hong, N. J. Majaj, R. Rajalingham, E. B. Issa, K. Kar, P. Bashivan, J. Prescott-Roy, F. Geiger, K. Schmidt, D. L. K. Yamins, and J. J. DiCarlo. Brain-score: Which artificial neural network for object recognition is most brain-like? *bioRxiv preprint*, 2018.

[28] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *ICCV*, 2017.

[29] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale imagerecognition. In *ICLR*, 2015.

[30] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. Intriguing properties of neural networks. In *ICLR*, 2014.

[31] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016.

[32] D. Ulyanov, A. Vedaldi, and V. Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv:1607.08022*, 2016.

[33] D. Warde-Farley and Y. Bengio. Improving generative adversarial networks with denoising feature matching. In *ICLR*, 2017.

[34] H. Wen, K. Han, J. Shi, Y. Zhang, E. Culurciello, and Z. Liu. Deep predictive coding network for object recognition. In *ICML*, 2018.

[35] D. L. K. Yamins and J. J. DiCarlo. Using goal-driven deep learning models to understand sensory cortex. *Nature Neuroscience*, 2016.

[36] A. R. Zamir, T.-L. Wu, L. Sun, W. B. Shen, B. E. Shi, J. Malik, and S. Savarese. Feedback networks. In *CVPR*, 2017.

Supplementary material

A Deconvolutional Generative Model

A.1 Generative model

We choose the deconvolutional generative model (DGM) [24] as the generative feedback in CNN-F. The graphical model of the DGM is shown in Figure 2 (middle). The DGM has the same architecture as CNN and generates images from high level to low level. Since low level features usually have higher dimension than high level features, the DGM introduces latent variables at each level to account for uncertainty in the generation process.

Let $y \in \mathbb{R}^K$ be label, K is the number of classes. Let $x \in \mathbb{R}^n$ be image and $h \in \mathbb{R}^m$ be encoded features of x after k convolutional layers. In a DGM with L layers in total, $g(\ell) \in \mathbb{R}^{C \times H \times W}$ denotes generated feature map at layer ℓ, and $z(\ell) \in \mathbb{R}^{C \times H \times W}$ denotes latent variables at layer ℓ. We use z_R and z_P to denote latent variables at a layer followed by ReLU and $\mathrm{MaxPool}$ respectively. In addition, we use $(\cdot)^{(i)}$ to denote the ith entry in a tensor. Let $W(\ell)$ and $b(\ell)$ be the weight parameters and bias parameters at layer ℓ in the DGM. We use $(\cdot)(*^\intercal)$ to denote deconvolutional transpose in deconvolutional layers and $(\cdot)^\intercal$ to denote matrix transpose in fully connected layers. In addition, we use $(\cdot)_\uparrow$ and $(\cdot)_\downarrow$ to denote upsampling and downsampling respectively. The generation process in the DGM is as follows:

$$y \sim p(y) \tag{6}$$

$$g(L-1) = W(L)^\intercal y \tag{7}$$

$$z_P(L-1)^{(i)} \sim \text{Ber}\left(\frac{e^{b(L-1)\cdot g(L-1)_\uparrow^{(i)}}}{e^{b(L-1)\cdot g(L-1)_\uparrow^{(i)}}+1}\right) \tag{8}$$

$$g(L-2) = W(L-1)(*^\intercal)\{g(L-1)_\uparrow \odot z_P(L-1)\} \tag{9}$$

$$\vdots$$

$$z_R(\ell)^{(i)} \sim \text{Ber}\left(\frac{e^{b(\ell)\cdot g(\ell)^{(i)}}}{e^{b(\ell)\cdot g(\ell)^{(i)}}+1}\right) \tag{10}$$

$$g(\ell-1) = W(\ell)(*^\intercal)\{z_R(\ell) \odot g(\ell)\} \tag{11}$$

$$\vdots$$

$$x \sim \mathcal{N}(g(0), \text{diag}(\sigma^2)) \tag{12}$$

In the above generation process, we generate all the way to the image level. If we choose to stop at layer k to generate image features h, the final generation step is $h \sim \mathcal{N}(g(k), \text{diag}(\sigma^2))$ instead of 12.

The joint distribution of latent variables from layer 1 to L conditioning on y is

$$\begin{aligned} p(\{z(\ell)\}_{\ell=1:L}|y) &= p(z(L)|y)\Pi_{\ell=1}^{L-1} p(z(\ell)|\{z(k)\}_{k\geq\ell}, y) \\ &= \text{Softmax}\left(\sum_{\ell=1}^{L} \langle b(\ell), z(\ell) \odot g(\ell)\rangle\right) \end{aligned} \tag{13}$$

where $\text{Softmax}(\eta) = \frac{\exp(\eta)}{\sum_\eta \exp(\eta)}$.

A.2 Proof for Theorem 2.1.B

In this section, we provide proofs for 2.1.B. In the proofs, we use f to denote the feedforward feature map after convolutional layer in the CNN of the same architecture as the DGM, and use $(\cdot)_a$ to denote layers after nonlinear operators. Let v be the logits output from fully-connected layer of the CNN. Without loss of generality, we consider a DGM that has the following architecture. We list the

corresponding feedforward feature maps on the left column:

$$
\begin{array}{rll}
 & & g(0) = W(1)(*^{\intercal})g_a(1) \\
\text{Conv} & f(1) = W(1) * x + b(1) & g_a(1) = g(1) \odot z_R(1) \\
\text{ReLU} & f_a(1) = \sigma_{\text{AdaReLU}}(f(1)) & g(1) = W(2)(*^{\intercal})g_a(2) \\
\text{Conv} & f(2) = W(2) * f_a(1) + b(2) & g_a(2) = g(2)_{\uparrow} \odot z_P(2) \\
\text{Pooling} & f_a(2) = \sigma_{\text{AdaPool}}(f(2)) & g(2) = W(3)^{\intercal} v \\
\text{FC} & v = W(3) f_a(2) &
\end{array}
$$

We prove Theorem 2.1.B which states that CNN with σ_{AdaReLU} and σ_{AdaPool} is the generative classifier derived from the DGM by proving Lemma A.1 first.

Definition A.1. σ_{AdaReLU} and σ_{AdaPool} are nonlinear operators that adaptively choose how to activate the feedforward feature map based on the sign of the feedback feature map.

$$
\sigma_{\text{AdaReLU}}(f) = \begin{cases} \sigma_{\text{ReLU}}(f), & \text{if } g \geq 0 \\ \sigma_{\text{ReLU}}(-f), & \text{if } g < 0 \end{cases} \qquad \sigma_{\text{AdaPool}}(f) = \begin{cases} \sigma_{\text{MaxPool}}(f), & \text{if } g \geq 0 \\ -\sigma_{\text{MaxPool}}(-f), & \text{if } g < 0 \end{cases} \tag{14}
$$

Definition A.2 (generative classifier)**.** Let v be the logits output of a CNN, and $p(x, y, z)$ be the joint distribution specified by a generative model. A CNN is a generative classifier of a generative model if $\text{Softmax}(v) = p(y|x, z)$.

Lemma A.1. *Let y be the label and x be the image. v is the logits output of the CNN that has the same architecture and parameters as the DGM. $g(0)$ is the generated image from the DGM. α is a constant. $\eta(y, z) = \sum_{\ell=1}^{L} \langle b(\ell), z(\ell) \odot g(\ell) \rangle$. Then we have*

$$
\alpha y^{\intercal} v = g(0)^{\intercal} x + \eta(y, z) \tag{15}
$$

Proof.

$$
\begin{aligned}
& g(0)^{\intercal} x + \eta(y, z) \\
=& \{W(1)(*^{\intercal})\{g(1) \odot z_R(1)\}\}^{\intercal} x + (z_R(1) \odot g(1))^{\intercal} b(1) + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) \\
=& (z_R(1) \odot g(1))^{\intercal} \{W(1)(*^{\intercal}) x + b(1)\} + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) \\
=& g(1)^{\intercal} (z_R(1) \odot f(1)) + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) \\
=& \{W(2)(*^{\intercal})\{g(2)_{\uparrow} \odot z_P(2)\}\}^{\intercal} (z_R(1) \odot f(1)) + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) \\
=& \{g(2)_{\uparrow} \odot z_P(2)\}^{\intercal} \{W(2) * (z_R(1) \odot f(1)) + b(2)\} \\
=& (W(3)^{\intercal} y)_{\uparrow}^{\intercal} \{z_P(2) \odot f(2)\} \\
=& \alpha (W(3)^{\intercal} y)^{\intercal} (z_P(2) \odot f(2))_{\downarrow} \\
=& \alpha y^{\intercal} W(3) (z_P(2) \odot f(2))_{\downarrow} \\
=& \alpha y^{\intercal} v
\end{aligned}
$$

□

Remark. *Lemma A.1 shows that logits output from the corresponding CNN of the DGM is proportional to the inner product of generated image and input image plus $\eta(y, z)$. Recall from 12, since the DGM assumes x to follow a Gaussian distribution centered at $g(0)$, the inner product between $g(0)$ and x is related to $\log p(x|y, z)$. Recall from equation 13 that conditionoal distribution of latent variables in the DGM is parameterized by $\eta(y, z)$. Using these insights, we can use Lemma A.1 to show that CNN performs Bayesian inference in the DGM.*
In the proof, the fully-connected layer applies a linear transformation to the input without any bias added. For fully-connected layer with bias term, we modify $\eta(y, z)$ to $\eta'(y, z)$:

$$
\eta'(y, z) = \eta(y, z) + y^{\intercal} b(3)
$$

The logits are computed by

$$
v = W(3)(f(2) \odot z(2)) + b(3)
$$

Following a very similar proof as of Lemma A.1, we can show that

$$
\alpha y^{\intercal} v = g^{\intercal}(0) + \eta'(y, z) \tag{16}
$$

With Lemma A.1, we can prove Theorem 2.1.B.

Assumption A.1. *The generated image $g(0)$ from the DGM has a constant ℓ_2 norm: $||g(0)||_2^2 =$ const.*

Assumption A.2. *Prior distribution on the label is a uniform distribution:* $p(y) = const.$

Assumption A.3. *Normalization factor in* $p(z|y)$ *for each category is constant:* $\sum_z e^{\eta(y,z)} = const.$

Theorem (Theorem 2.1.B)**.** *Under assumptions A.1, A.2 and A.3, CNN with* $\sigma_{AdaReLU}$ *and* $\sigma_{AdaPool}$ *is the generative classifier derived from the DGM.*

Proof. We use $p(x, y, z)$ to denote the joint distribution specified by the DGM. In addition, we use $q(y|x, z)$ to denote the Softmax output from the CNN, i.e. $q(y|x, z) = \frac{y^\intercal e^v}{\sum_{i=1}^K e^{v^{(i)}}}$. To simplify the notation, we use z instead of $\{z(\ell)\}_{\ell=1:L}$ to denote latent variables across layers.

$$
\begin{aligned}
&\log p(y|x, z) \\
=&\log p(y, x, z) - \log p(x, z) \\
=&\log p(x|y, z) + \log p(z|y) + \log p(y) - \log p(x, z) \\
=&\log p(x|y, z) + \log p(z|y) + \text{const.} && (*) \\
=&-\frac{1}{2\sigma^2}||x - g(0)||_2^2 + \log \text{Softmax}(\eta(y, z)) + \text{const.} \\
=&\frac{1}{\sigma^2} g(0)^\intercal x + \log \text{Softmax}(\eta(y, z)) + \text{const.} && \text{(Assumption A.1)} \\
=&\frac{1}{\sigma^2} g(0)^\intercal x + \log \frac{e^{\eta(y,z)}}{\sum_z e^{\eta(y,z)}} + \text{const.} \\
=&\frac{1}{\sigma^2} g(0)^\intercal x + \eta(y, z) + \text{const.} && \text{(Assumption A.3)} \\
=&\alpha y^\intercal v + \text{const.} && \text{(Lemma A.1)} \\
=&\alpha(\log q(y|x, z) + \log \sum_{i=1}^K e^{v^{(i)}}) + \text{const.} \\
=&\alpha \log q(y|x, z) + \text{const.} && (**)
\end{aligned}
$$

We obtain line $(*)$ for the following reasons: $\log p(y) = \text{const.}$ according to Assumption A.2, and $\log p(x, z) = \text{const.}$ because only y is variable, x and z are given. We obtained line $(**)$ because given x and z, the logits output are fixed. Therefore, $\log \sum_{i=1}^K e^{v^{(i)}} = \text{const.}$. Take exponential on both sides of the above equation, we have

$$p(y|x, z) = \beta q(y|x, z) \tag{17}$$

where β is a scale factor.

Since both $q(y|x, z)$ and $p(y|x, z)$ are distributions, we have $\sum_y p(y|x, z) = 1$ and $\sum_y q(y|x, z) = 1$. Summing over y on both sides of equation 17, we have $\beta = 1$. Therefore, we have $q(y|x, z) = p(y|x, z)$. □

We have proved that CNN with σ_{AdaReLU} and σ_{AdaPool} is the generative classifier derived from the DGM that generates to layer 0. In fact, we can extend the results to all intermediate layers in the DGM.

Assumption A.4. *Each generated layer in the DGM has a constant* ℓ_2 *norm:* $||g(\ell)||_2^2 = const., \ell = 1, \dots, L.$

Assumption A.5. *Normalization factor in* $p(z|y)$ *up to each layer is constant:* $\sum_z e^{\eta(y, \{z(j)\}_{j=\ell:L})} = const., \ell = 1, \dots, L.$

Corollary A.1.1. *Under assumptions A.4, A.2 and A.5, CNN with* $\sigma_{AdaReLU}$ *and* $\sigma_{AdaPool}$ *starting with an intermediate layer is the generative classifier derived from the DGM that generates to the same intermediate layer:*

$$q(y|f(\ell), \{z(j)\}_{j=\ell:L}) = p(y|f(\ell), \{z(j)\}_{j=\ell:L}), \quad \ell = 1, \dots, L.$$

A.3 Proof for Theorem 2.1.C

In this section, we provide proofs for 2.1.C. In the proofs, we inherit the notations that we use for proving 2.1.B. Without loss of generality, we consider a DGM that has the same architecture as the one we use to prove 2.1.B.

Theorem (Theorem 2.1.C). *Under assumptions A.1, A.2 and A.3, MAP estimate of $z(\ell)$ conditioned on x, y and $\{z(j)\}_{j\neq\ell}$ in the DGM is:*

$$\hat{z}_R(\ell) = \mathbb{1}(\sigma_{AdaReLU}(f(\ell)) \geq 0) \tag{18}$$

$$\hat{z}_P(\ell) = \mathbb{1}(g(\ell) \geq 0) \odot \arg\max_{r\times r}(f(\ell)) + \mathbb{1}(g(\ell) < 0) \odot \arg\min_{r\times r}(f(\ell)) \tag{19}$$

Proof.

$$\begin{aligned}
&\arg\max_{z(\ell)} \log p(z(\ell)|\{z(j)\}_{j\neq\ell}, x, y) \\
&= \arg\max_{z(\ell)} \log p(\{z(j)\}_{j=1:L}, x, y) \\
&= \arg\max_{z(\ell)} \log p(x|y, \{z(j)\}_{j=1:L}) + \log p(\{z(j)\}_{j=1:L}|y) + \log p(y) \\
&= \arg\max_{z(\ell)} \log p(x|y, \{z(j)\}_{j=1:L}) + \eta(y, z) + \text{const.} && \text{(Assumption A.3 and A.2)} \\
&= \arg\max_{z(\ell)} \frac{1}{\sigma^2} g(0)^\intercal x + \eta(y, z) + \text{const.} && \text{(Assumption A.1)}
\end{aligned}$$

Using Lemma A.1, the MAP estimate of $z_R(\ell)$ is:

$$\begin{aligned}
\hat{z}_R(\ell) &= \arg\max_{z_R(\ell)} (z_R(\ell) \odot g(\ell))^\intercal f(\ell) \\
&= \mathbb{1}(\sigma_{\text{AdaReLU}}(f(\ell)) \geq 0)
\end{aligned}$$

The MAP estimate of $z_P(\ell)$ is:

$$\begin{aligned}
\hat{z}_P(\ell) &= \arg\max_{z_P(\ell)} (z_P(\ell) \odot g(\ell)_\uparrow)^\intercal f(\ell) \\
&= \mathbb{1}(g(\ell) \geq 0) \odot \arg\max_{r\times r}(f(\ell)) + \mathbb{1}(g(\ell) < 0) \odot \arg\min_{r\times r}(f(\ell))
\end{aligned}$$

□

A.4 Incorporating instance normalization in the DGM

Inspired by the constant norm assumptions (Assumption A.1 and A.4), we incorporate instance normalization into the DGM. We use $\overline{(\cdot)} = \frac{(\cdot)}{||\cdot||_2}$ to denote instance normalization, and $(\cdot)_n$ to denote layers after instance normalization. In this section, we prove that with instance normalization, CNN is still the generative classifier derived from the DGM. Without loss of generality, we consider a DGM that has the following architecture. We list the corresponding feedforward feature maps on the left column:

$$\begin{aligned}
& && g(0) = W(1)(*^\intercal) g_a(1) \\
\text{Conv} \quad & f(1) = W(1) * \overline{x} && g_a(1) = g_n(1) \odot z_R(1) \\
\text{Norm} \quad & f_n(1) = \overline{f(1)} && g_n(1) = \overline{g(1)} \\
\text{ReLU} \quad & f_a(1) = \sigma_{\text{AdaReLU}}(f_n(1) + b(1)) && g(1) = W(2)(*^\intercal) g_a(2) \\
\text{Conv} \quad & f(2) = W(2) * f_a(1) && g_a(2) = g_n(2)_\uparrow \odot z_P(2) \\
\text{Norm} \quad & f_n(2) = \overline{f(2)} && g_n(2) = \overline{g(2)} \\
\text{Pooling} \quad & f_a(2) = \sigma_{\text{AdaPool}}(f_n(2) + b(2)) && g(2) = W(3)^\intercal v \\
\text{FC} \quad & v = W(3) f_a(2) &&
\end{aligned}$$

Assumption A.6. *Feedforward feature maps and feedback feature maps have the same ℓ_2 norm:*

$$||g(\ell)||_2 = ||f(\ell)||_2, \ell = 1, \ldots, L$$
$$||g(0)||_2 = ||x||_2$$

Lemma A.2. *Let y be the label and x be the image. v is the logits output of the CNN that has the same architecture and parameters as the DGM. $g(0)$ is the generated image from the DGM, and $\overline{g(0)}$ is normalized $g(0)$ by ℓ_2 norm. α is a constant. $\eta(y, z) = \sum_{\ell=1}^{L} \langle b(\ell), z(\ell) \odot g(\ell)\rangle$. Then we have*

$$\alpha y^\intercal v = \overline{g(0)}^\intercal x + \eta(y, z) \tag{20}$$

Proof.

$$
\begin{aligned}
&\overline{g(0)}^{\intercal} x + \eta(y, z) \\
=&\{W(1)(*^{\intercal})\{g_n(1) \odot z_R(1)\}\}^{\intercal} \frac{x}{||g(0)||_2} + (z_R(1) \odot g(1))^{\intercal} b(1) + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) \\
=&(z_R(1) \odot g_n(1))^{\intercal} \{W(1)(*^{\intercal})\overline{x}\} + (z_R(1) \odot g(1))^{\intercal} b(1) + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) && \text{(Assumption A.6)} \\
=&g(1)^{\intercal} \{z_R(1) \odot (f_n(1) + b(1))\} + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) && \text{(Assumption A.6)} \\
=&\{W(2)(*^{\intercal})\{g_n(2)_{\uparrow} \odot z_P(2)\}\}^{\intercal} (z_R(1) \odot f(1)) + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) \\
=&\{g_n(2)_{\uparrow} \odot z_P(2)\}^{\intercal} \{W(2) * (z_R(1) \odot f(1))\} + (z_P(2) \odot g(2)_{\uparrow})^{\intercal} b(2) \\
=&g(2)^{\intercal} \{z_P(2) \odot (f_n(2) + b(2))\} && \text{(Assumption A.6)} \\
=&(W(3)^{\intercal} y)_{\uparrow}^{\intercal} \{z_P(2) \odot f(2)\} \\
=&\alpha (W(3)^{\intercal} y)^{\intercal} (z_P(2) \odot f(2))_{\downarrow} \\
=&\alpha y^{\intercal} W(3) (z_P(2) \odot f(2))_{\downarrow} \\
=&\alpha y^{\intercal} v
\end{aligned}
$$

□

Theorem A.3. *Under assumptions A.6 and A.2, CNN with $\sigma_{AdaReLU}$ and $\sigma_{AdaPool}$ and instance normalization is the generative classifier derived from the DGM with instance normalization.*

Proof. The proof of Theorem A.3 is very similar to that of Theorem 2.1 using Lemma A.2. Therefore, we omit the detailed proof here. □

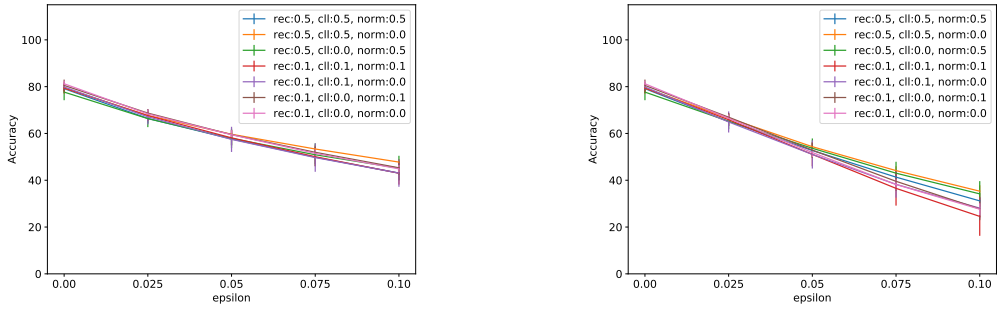
Remark. *The instance normalization that we incorporate into the DGM is not the same as the instance normalization that people typically used for image stylization [32]. The conventional instance normalization computes output y from input x as $y = \frac{x-\mu(x)}{\sigma(x)}$, where μ and σ stands for mean and standard deviation respectively. Our instance normalization does not subtract the mean of the input and divides the input by its ℓ_2 norm to make it have constant ℓ_2 norm.*

B Experiment details

B.1 Training details

Experimental setup We use the following architecture to train CNN-F on MNIST and Fashion-MNIST: Conv2d(32, 3× 3), Instancenorm, AdaReLU, Conv2d(64, 3× 3), Instancenorm, AdaPool, Reshape, FC(128), AdaReLU, FC(10). The instance normalization layer we use is described in Appendix A.4. All the images are scaled between $[-1, +1]$ before training. We train both CNN and CNN-F with Adam [14]. We set weight decay to 0.0005. For standard training and adversarial training, we use learning rate of 0.001 and 0.0001 respectively.
For standard training, we train for 30 epochs. We have training losses as listed in Table 1. We use $\mathcal{L}_{Recon}$ to denote reconstruction loss and $\mathcal{L}_{Latent}$ to denote conditional latent likelihood loss. In addition, we have norm matching loss $\mathcal{L}_{Norm}$ when using instance normalization. Norm matching loss is designed to enforce Assumption A.6. Specifically, we have $\mathcal{L}_{Norm} = \sum_{\ell=1}^{L} \text{abs}(||f(\ell)||_2 - ||g(\ell)||_2)$, where abs stands for absolute value.
For adversarial training, we train for 90 epochs. We have Cross entropy loss on adversarial samples. In addition, we have reconstruction loss between adversarial samples and the corresponding clean samples in CNN-F. We use the projected gradient descent (PGD) method to generate adversarial samples within L_∞-norm constraint, and denote the maximum L_∞-norm between adversarial images and clean images as ϵ. The step size in PGD attack is set to 0.02 for $\epsilon \leq 0.6$ and to 0.03 for $\epsilon > 0.6$. Since we preprocess images to be within range $[-1, +1]$, the values of ϵ that we report in this paper are half of their actual values to show a relative perturbation strength with respect to range $[0, 1]$.

Hyper-parameters We use α_{rec}, α_{cll}, and α_{norm} to denote coefficients on $\mathcal{L}_{Recon}$, $\mathcal{L}_{Latent}$ and $\mathcal{L}_{Norm}$ respectively. We fix the coefficient for cross entropy loss to be 1.0 for all configurations. Figure 10 shows the influence of hyper-parameter configurations on adversarial accuracy. As we can see, hyper parameter configurations do not have large influence on adversarial robustness. The results we presented in this paper are generated using $\alpha_{rec} = 0.1, \alpha_{cll} = 0.0, \alpha_{norm} = 0.0$.



(a) Standard training. Testing w/ FGSM.

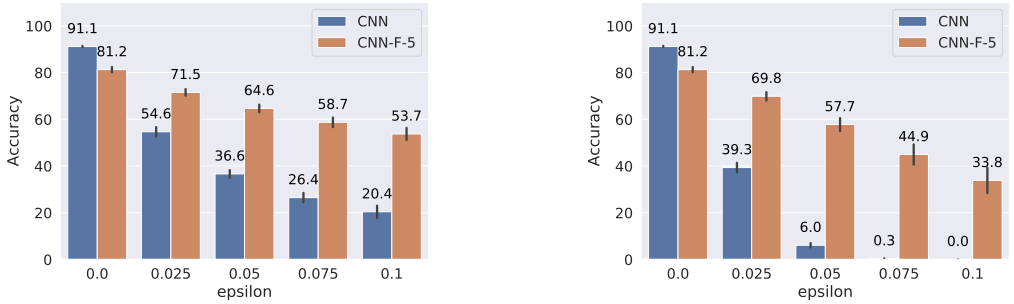
(b) Standard training. Testing w/ PGD-40.

Figure 10: **Effect of hyper-parameters on adversarial robustness.** We train CNN-F-5 on Fashion-MNIST under standard training. Each accuracy is averaged over 5 runs and the error bar indicates standard deviation.

B.2 End-to-end BPDA attack

We explore various ways to attack the CNN-F. First, we attempt to perform an end-to-end Backward Pass Differentiable Approximation (BPDA) [1] attack on CNN-F. Second, we attack the first feedforward pass of CNN-F. This is equivalent to attacking a CNN with the same parameters and attempting to transfer the attack from the CNN over to the CNN-F. We call this method transfer attack for short. Since transfer attack is more effective than end-to-end BPDA attack, we report the adversarial robustness against transfer attack in the main text.

There are two reasons for the degraded the effectiveness of end-to-end BPDA attack. Since σ_{AdaReLU} and σ_{AdaPool} in the CNN-F are non-differentiable, we need to use BPDA to approximate the gradient during back propagation in the end-to-end attack. Furthermore, to perform the end-to-end attack, we need to back propagate through unrolled CNN-F, which is k times deeper than the corresponding CNN, where k is the number of iterations during evaluation. Figure 11 shows the results of end-to-end BPDA attack. CNN-F-5 significantly improves the robustness of CNN.



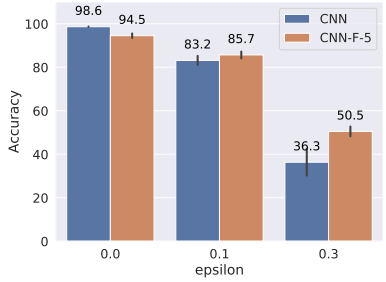
(a) Standard training. Testing w/ FGSM.

(b) Standard training. Testing w/ PGD-40.

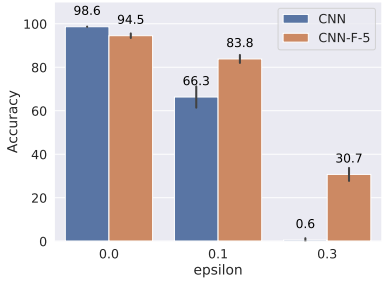
Figure 11: **Adversarial robustness on Fashion-MNIST against end-to-end BPDA attack.** CNN-F-k stands for CNN-F trained with k iterations; PGD-c stands for a PGD attack with c steps. CNN-F achieves higher accuracy on MNIST than CNN for under both standard training and adversarial training. Each accuracy is averaged over 4 runs and the error bar indicates standard deviation. We use the following hyper-parameter configuration: $\alpha_{Recon} = 0.5, \alpha_{Latent} = 0.5, \alpha_{Norm} = 0.5$.

B.3 Adversarial robustness on MNIST

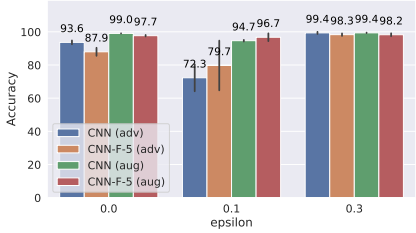
In addition to Fashion-MNIST, we also evaluated the robustness of CNN-F on MNIST. Figure 12 shows the results. CNN-F-5 improves the robustness of CNN under both standard training and adversarial training. Furthermore, when adversarially trained with PGD-7 attack, CNN-F-5 generalizes much better to PGD-40 attack compared to CNN (Figure 12d).



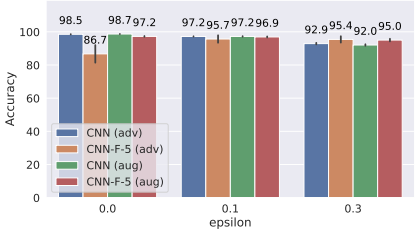
(a) Standard training. Testing w/ FGSM.



(b) Standard training. Testing w/ PGD-40.



(c) Adv. training w/ FGSM ($\epsilon = 0.3$). Testing w/ FGSM.

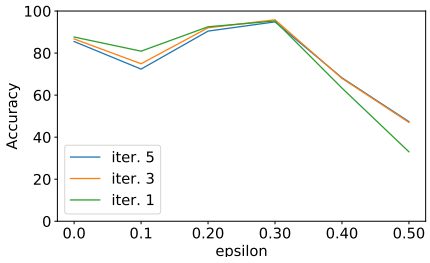


(d) Adv. training w/ PGD-40($\epsilon = 0.3$). Testing w/ PGD-40.

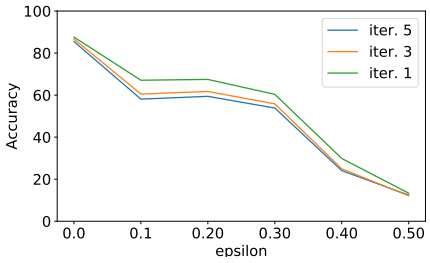
Figure 12: **Adversarial robustness of CNN-F on MNIST.** CNN-F-k stands for CNN-F trained with k iterations; PGD-c stands for a PGD attack with c steps. CNN-F achieves higher accuracy on MNIST than CNN for under both standard training and adversarial training. Each accuracy is averaged over 5 runs and the error bar indicates standard deviation.

B.4 Effect of number of iterations

The optimal number of iterations to achieve the highest accuracy varies for different types of images. Empirically, we find that more iterations are needed for harder attack method (such as PGD-40) and larger perturbation magnitude and vice-versa. This motivates future work of adaptively choosing number of iterations based on input images.



(a) Training w/ PGD-40. Testing w/ PGD-40.



(b) Training w/ PGD-40. Testing w/ FGSM.

Figure 13: **Effect of number of iterations on adversarial robustness.** More iterations are needed for harder attack with larger perturbation magnitude.

B.5 Neural predictivity

B.5.1 Training details

The CNN-F and CNN models both use the VGG-16 architecture. The CNN model used pre-trained weights from the the PyTorch torchvision pre-trained model set [3]. The CNN-F model was also

[3] https://pytorch.org/docs/stable/torchvision/models.html

initialized with these pre-trained weights. The CNN-F was trained for 200,000 iterations with a batch size of 128 images. These images were normalized to zero mean and unit variance with the mean $\mu = (0.485, 0.456, 0.406)$ and the standard deviation $\sigma = (0.229, 0.224, 0.225)$. Random horizontal flips was the only augmentation used. Optimization used stochastic gradient descent with a momentum of 0.9 and a learning rate of 0.005. The learning rate was decreased by 10% every 100,000 iterations.

B.5.2 Neural correspondence benchmark

To calculate the neural predictivity, 2760 images containing a single object on a random natural background were presented centrally to passively fixated monkeys for 100 ms. The neural responses were recorded from 88 V4 sites and 168 IT sites. A regression model was constructed for each neuron using 90% of image responses and tested on the remaining 10% in a 10-fold cross-validation strategy. The median Pearson's r between the predicted and actual response constituted the final neural predictivity score [27].